WeatherSafe: Technical Architecture and Feasibility Research

Headline findings

1. **A covert trigger is only dependable while the app is open on screen.** Android restricts starting location, microphone and camera foreground services from the background, and neither platform lets an app listen to the microphone in the background without visible indicators. The MVP trigger should be a foreground gesture, not a hidden background listener. [Sourced + Inference]
2. **'No backend' is not realistic for alerts.** A contact's phone cannot be reliably woken for a new alert by a direct peer-to-peer link alone; push notifications go through Google (FCM) and Apple (APNs) servers, and WebRTC itself needs a signalling server and often a TURN relay. [Sourced + Inference]
3. **iOS location in the background shows a visible indicator** unless the user grants the stronger 'Always' permission, which the user can decline. This directly affects covertness. [Sourced]
4. **Framework choice is not a performance question.** Recent comparisons say the gap between Flutter and React Native has narrowed. The deciding factor is whether the plugins we need are maintained, which needs a short spike in each. [Sourced, low-quality sources]
5. **Machine learning is not needed for the first vertical slice.** Simple threshold rules can detect a tap pattern or shake. LiteRT (the successor to TensorFlow Lite) is a sensible stretch goal. [Inference]

Recommended direction (provisional)

- **MVP:** foreground hidden gesture, location sharing started by that gesture, a small relay server plus push notifications to alert the contact, and a contact dashboard inside the app.
- **Stretch:** WebRTC data channel for live updates, an accelerometer gesture model using LiteRT, and the iOS 'Always' location option.
- **Deferred or out of scope for the build:** background codeword listening, automatic escalation to emergency services, evidence recording.

2. Scope and method

This document answers the eight research tasks assigned to part E. For each, it gives the question, the evidence found, what that means for WeatherSafe, a decision or recommendation, new risks, and whether the topic is MVP, stretch or future work. This follows the 'questions every research area should answer' list in the team brief (section 10 of the brief).

Boundary with part F (David): I cover whether it can be built; David covers whether it is secure and lawful. Section 11 lists the points where we must agree on things.

3. Task 1: Flutter versus React Native

Question. Which framework better supports native device capabilities, performance, platform compatibility and development complexity for WeatherSafe?

Evidence

- Several 2026 comparison articles agree that both frameworks are mature and production-ready. They say React Native's 'New Architecture' is now the production default and has narrowed the performance gap with Flutter. [S15, S16, S17]
- Flutter draws its own interface with its own rendering engine (Impeller), which the sources say helps with animation-heavy, custom interfaces. React Native uses the platform's native components. [S16, S17]
- React Native has a larger ecosystem and a larger pool of JavaScript developers; Flutter is described as having stronger typing and a more structured documentation set. One source says React Native is better for deep hardware integration and on-device ML. [S15, S16, S17]
- One source reports React Native using less memory because it shares the native runtime, while Flutter bundles its own engine. [S16]

Source quality

Low to medium. These are blog posts from agencies and vendors with commercial and search-ranking motives, and the benchmark claims do not give methods. I use them only for the general picture and not for any decision on their own.

What it means for WeatherSafe

[Inference] The performance difference is unlikely to matter for a weather screen plus a map. The real risks are plugin-level: background location, foreground service control, push notification handling, sensor access, WebRTC and ML. Both frameworks will probably still need some native Kotlin and Swift code for the trigger and the location service. A custom, polished weather interface is a point in Flutter's favour; the team's existing language skills are a practical point in either direction (team to confirm).

Comparison framework (to be completed during the spike)

Criterion	Weight (agree with team)	Flutter	React Native	Evidence level
Background or foreground-service location	High	To test	To test	Plugin docs + spike
Push notifications (FCM and APNs)	High	To test	To test	Plugin docs + spike
Sensor access (accelerometer, touch handling)	High	To test	To test	Spike
WebRTC plugin maintained	Medium	To verify	To verify	Plugin repository activity
On-device ML (LiteRT) support	Low (stretch)	To verify	To verify	Plugin docs

Criterion	Weight (agree with team)	Flutter	React Native	Evidence level
Weather UI quality and development speed	Medium	To test	To test	Team trial
Team skills and learning time	Medium	Team to state	Team to state	Team input

Plugins to evaluate: the official Firebase plugins for each framework, a WebRTC plugin for each (for example flutter_webrtc and react-native-webrtc), and a background location plugin for each. I have not verified their current maintenance status; check last release date, open issues and supported OS versions.

Decision

Do not decide now. Run the same three-feature spike in both frameworks (a tap-pattern trigger, location while the app is minimised after a foreground trigger, receiving an FCM push on a second phone). Choose using the weighted table. If the result is a tie, choose the framework the team is most comfortable with. Scope: MVP decision, needed before sprint planning.

4. Task 2: Covert activation methods

Question. Which concealed triggers are feasible with sensors and gestures, and how do they perform?

Evidence

- Android 9 and higher does not allow microphone or camera access while the app is in the background, and Android recommends showing a real-time indicator whenever audio is being captured. [S6]
- Android 11 and later: a foreground service started while the app is in the background cannot access the microphone or camera, and cannot access location without background location permission. [S5]
- On iOS 14 and later, an orange dot appears in the status bar while the microphone is in use; on Android 12 and later a green dot appears. [S21] (secondary source)

What it means

[Inference] The brief's out-of-scope list already excludes always-on microphone monitoring. The platform rules reinforce that: a codeword listener that works while the phone is in a pocket would either be blocked or would show an indicator, which defeats the purpose. Triggers that work in the foreground, inside the weather interface, are feasible. I recommend the team does not try to evade the OS indicators; this is an ethics and app-store risk as well as a technical one, and David should be consulted.

Options compared

Trigger	Works in background?	Main risk	Needs ML?	Recommendation
Hidden touch pattern in the weather UI (long press on a specific element, or a short sequence of taps)	No, the app must be open	Discoverability; user must remember it	No	MVP
Shake or motion pattern read from the accelerometer	Limited; needs the app active or a background service	False activations while walking, running or on transport	Threshold rules first; ML later	Stretch
Spoken codeword	Not realistically (see evidence)	Indicators, consent, false positives, noise	Yes	Defer; keep as research finding
Hardware button presses	OS-level behaviour varies by platform and is largely not available to apps	Unreliable	No	[To verify] before ruling in or out

Prototype plan

1. **Spike A (tap pattern):** implement a hidden hotspot on the weather screen. Log every attempt. Measure how often the correct pattern is detected, how often normal use triggers it by accident, and how long it takes.
2. **Spike B (motion):** read the accelerometer, detect a defined shake with a simple threshold, and log false triggers during a 30-minute walk and a short bus or car journey.

Metrics

True-positive rate, false activations per hour of normal use, time from start of gesture to incident start, and user recall after one day (coordinate with Ben, who owns the interaction design and the cancel flow).

Decision

MVP trigger = foreground hidden touch pattern, with a confirmation or cancel window designed by Ben. Scope: MVP. Open risk: if the app must also react while the screen is locked, the whole trigger design changes.

5. Task 3: On-device ML frameworks (LiteRT and others)

Question. Which frameworks could recognise gestures or codewords locally, and is ML worth including?

Evidence

- LiteRT is Google's on-device runtime and the successor to TensorFlow Lite; it runs inference fully on-device on Android and iOS with Kotlin, Swift and C++ APIs. [S13, S14]
- LiteRT 2.x introduces a newer CompiledModel API with hardware acceleration, while the older Interpreter API (the TensorFlow Lite API) remains available. Migrating existing TensorFlow Lite code is described as needing few code changes because the method names are the same. [S13]
- Models can be converted from TensorFlow, PyTorch or JAX. [S13]

Options

Option	Fit for WeatherSafe	Effort	Notes
No ML: threshold and sequence rules	Good for a tap pattern or a single shake	Low	Enough for the vertical slice [Inference]
LiteRT custom accelerometer gesture model	Good for several gestures; learns patterns that rules cannot	High: needs labelled data from several people	Stretch
Platform-native ML (for example Core ML on iOS)	Possible; two separate implementations	High	[To verify]; likely redundant if LiteRT works on both
On-device keyword spotting	Constrained by the microphone limits in section 4	High	Defer

Decision

No ML in the MVP. For a stretch spike: record accelerometer windows from at least five volunteers (with consent), train a small classifier, convert it to LiteRT format, run it in the app and measure accuracy,

latency and battery use. Whether LiteRT can be called cleanly from Flutter or React Native is [To verify] and belongs in the framework spike.

6. Task 4: WebRTC and peer-to-peer communication

Question. Can WebRTC carry the alert and location data directly between phones, and what infrastructure is unavoidable?

Evidence

- WebRTC does not define how peers find each other; the signalling channel is up to the application, so a signalling server is needed. [S18]
- Peers use STUN to discover their public address and attempt a direct connection; when that fails (for example behind symmetric NATs or strict firewalls) a TURN server relays the traffic. A TURN relay adds latency and server bandwidth cost. [S18, S19]
- A TURN service vendor claims STUN alone is enough about 85% of the time. This is a commercial claim without method, so I do not use it as a planning figure. [S20]

What it means

[Inference] Even in the best case a WebRTC design needs: (a) a signalling server, (b) a TURN relay as fallback, and (c) a way to get the contact's phone to open the app and connect, which in practice means a push notification, which in turn needs FCM or APNs servers. Both users must also be online at the same time. So WebRTC cannot be presented as 'no cloud'. What it can offer is that the incident content (location updates) travels device to device and is not stored on our server.

Architecture options

	A: Relay and push	B: WebRTC data channel plus push	C: Hybrid
Servers needed	App server, FCM, APNs	Signalling server, TURN, FCM, APNs	All of B plus a relay fallback
Contact offline or app closed	Works through push, queued messages	Needs push, then connection setup	Works through push
Latency	Low to moderate	Low when direct, higher when relayed	Low to moderate
Server sees content?	Only if not end-to-end encrypted	Not when direct; TURN relays encrypted media or data	Depends on path
Complexity	Low	High	Highest
Fits MVP timeline	Yes	Unlikely	No

Decision

MVP uses option A, with payloads end-to-end encrypted by a scheme David specifies, so the server relays but cannot read incident data. WebRTC (option B or C) is a stretch comparison; the evidence does not support claiming 'fully peer-to-peer' or 'no backend'. This matches the brief's warning in section 7. Scope: MVP for A; stretch for B.

7. Task 5: Android and iOS platform restrictions

Question. What do Android and iOS allow for background execution, location, microphone and notifications?

Area	Android	iOS	Implication for WeatherSafe
Background execution	Since Android 12, apps cannot start foreground services from the background, with a few exceptions. [S3] Since Android 14, each foreground service must declare a type. Android 15 added time limits for some service types. [S2]	Background capabilities must be enabled in the project, and Apple advises considering carefully whether background location is needed. [S9]	Start the incident from the foreground, then let the service continue in the background. Check that the service type we need has no time limit.
Location	A foreground service that needs location cannot be started from the background unless the app has background location permission. [S3, S5] A foreground service shows a notification, and since Android 13 users can dismiss it. [S1]	'When in Use' keeps location running in the background while updates are active; the system will not relaunch the app if it has been terminated. 'Always' lets the system relaunch for some updates. [S10] Apps with 'When in Use' get a visible blue indicator in the status bar when using location in the background; apps with 'Always' can control it. [S11] The user may grant 'When in Use' even if 'Always' is requested, and the app can request 'Always' only once. [S10]	Covertness cost on iOS: either a blue indicator or reliance on 'Always'. Design for 'When in Use' first. Apple tells developers to inform the user when updates arrive in the background. [S9]
Microphone	Not available in the background on Android 9 and higher; indicators	An orange indicator dot appears when the microphone is in use	Background codeword listening is not viable. Any mic feature needs

Area	Android	iOS	Implication for WeatherSafe
	and a privacy dashboard exist on Android 12 and higher. [S6, S21]	(iOS 14 and later). [S21]	consent and review (part F).
Notifications	Doze and App Standby delay background work. A high-priority FCM message can wake the app to show a user-visible notification, and should only be used for messages that create one. [S7, S8]	Apple defines Time Sensitive and Critical interruption levels. Time Sensitive can break through Focus. Critical can also override the silent switch and requires an approved entitlement. The system lets users turn off Time Sensitive after the first one. [S12]	Alerts to the contact must use high priority (Android) and at least Time Sensitive (iOS), otherwise Focus or Doze can hide them. Whether Apple would approve Critical alerts for a student prototype is unknown [To verify].

Additional points

- [Inference] The covert user's own device should not show an obvious incident notification. The notification the OS requires for the location service is a design constraint for Ben to solve, for example a neutral weather-style notification.
- [To test] Behaviour differs between manufacturers' Android versions, which often kill background apps more aggressively. Test on at least two Android brands if possible.

Decision

Design the MVP around foreground-initiated incidents, high-priority push for the contact, and 'When in Use' location on iOS. Treat 'Always' permission and Critical alerts as stretch items.

8. Task 6: Real-time location sharing

Question. How could live location move from the user to the trusted contacts?

Evidence and constraints

- Android recommends starting a foreground service before the app goes to the background if the user started a continuing task, and asking for background location only where necessary. [S6]
- Android location foreground services are intended for long-running uses such as navigation and location sharing. [S4]
- On iOS, a background activity session is used to receive location updates in the background, and the system shows an indicator as described in section 7. [S9, S11]

Proposed flow (MVP)

1. The user performs the hidden gesture in the foreground.
2. The app starts the incident, starts a location service, and sends a start message to the relay server.
3. The server sends a high-priority push to each authorised contact.
4. The contact opens the app and sees a dashboard with the latest location, timestamp, accuracy and incident status.
5. Location updates continue at a fixed interval until the user ends the incident, then they stop. This follows the brief's requirement that sharing stops when the incident ends.

Design questions to settle in the spike

- **Update interval and battery:** test several intervals (for example 5, 15 and 30 seconds) and measure battery and freshness [To test].
- **Stale data:** show the age of the last update and the accuracy circle on the contact's map; never show an old position as current.
- **Poor signal:** queue updates on the phone and send the newest when the connection returns.
- **Contact without the app:** out of MVP scope; a fallback (such as a text message) is a later question [To verify what each platform allows].
- **Map on the contact dashboard:** choose a map provider and check licensing and cost [To verify].

Decision

MVP = periodic location updates through the relay (option A), initiated from the foreground. Live peer-to-peer streaming is a stretch item.

9. Task 7: Battery, latency, compatibility and offline behaviour

Question. What are the battery cost, latency, device coverage and offline behaviour?

This cannot be answered from documentation alone. The table below is the test plan; the results columns are intentionally empty until tests are run.

Test scenarios

1. Idle weather app open for 10 minutes (baseline).
2. Incident active for 30 minutes with the screen off.
3. Incident active while walking outdoors.
4. Alert sent on good Wi-Fi, on mobile data, and on weak signal.
5. Contact's phone idle, locked, and in Do Not Disturb or Focus.
6. User phone loses connection mid-incident, then regains it.

Measurements

Metric	How to measure	Proposed target (to agree with team)	Android result	iOS result
Trigger-to-contact alert latency	Timestamp the trigger and the push arrival on both phones	Under 10 seconds on a good connection		
Battery drain per hour during an incident	Android battery statistics; Xcode energy gauge or Instruments on iOS [to verify tools]	Agree an acceptable limit		
Location freshness at the contact	Compare displayed age with real time	Updates no older than the chosen interval plus delivery time		
False activations per hour	Log during normal use	As close to zero as practical		
Offline recovery	Time to resume after reconnecting	Within one interval		
Device coverage	Run the slice on oldest and newest available phones	Agree minimum OS versions		

Target numbers above are my proposals and are not sourced; treat them as starting points for discussion.

Decision

Carry out scenarios 2, 4 and 5 first because they test the riskiest assumption: that alerts reach a locked contact's phone quickly.

10. Task 8: Recommended technology stack and deferred features

Provisional stack

Layer	Recommendation	Status
Mobile app	Flutter or React Native (decide after the spike)	Open

Layer	Recommendation	Status
Disguise UI	Weather screen with a hidden gesture area	MVP
Trigger	Foreground hidden touch pattern; motion gesture as stretch	MVP / stretch
Location	Platform location APIs through a plugin, with a native service where required	MVP
Alert and data transport	Small relay server (HTTPS or WebSocket) plus FCM and APNs push	MVP
End-to-end encryption and pairing	Specified by David (part F)	Joint
Contact dashboard	Inside the same app: map, status, timeline	MVP
ML	LiteRT for a gesture model	Stretch
WebRTC	Data channel with signalling and TURN	Stretch

Architecture diagram (text version)

```
[User phone: weather app]
   |  hidden gesture (foreground)
   v
[Incident starts] --> [Location service] --encrypted updates--> [Relay
server] --> [Contact phone]
                                                    |                 ^
                                                    v                 |
                                                [FCM / APNs push] -------+
                                                    |
                                            Contact opens app: dashboard
                                            (map, status, timeline)
```

A clean diagram should be drawn from this in a diagramming tool for the final report.

Features to simplify or defer

Feature	Reason
Background codeword or continuous audio detection	Blocked or indicator-visible on both platforms; consent and ethics risk (section 4)
Multiple ML gestures	No data or model yet; rules cover the slice

Feature	Reason
Fully peer-to-peer, no-backend claim	Signalling, TURN and push infrastructure still required (section 6)
Always-on iOS location and Critical alerts	Visible indicator trade-off; needs Apple approval
Evidence recording, fake call, geofenced monitoring	Not needed for the first vertical slice; platform limits apply
Automatic emergency-service escalation	Out of scope in the brief

11. Interfaces with part F (security and privacy)

Points to agree with David:

1. End-to-end encryption of location and alert payloads, and who holds the keys.
2. How a trusted contact is paired and verified, and how spoofed alerts are rejected.
3. What the relay server stores and for how long (the brief asks for data minimisation).
4. How push tokens and contact identities are protected.
5. TURN credential handling, if WebRTC is used.
6. Consent, indicators and ethics of any microphone use; the position that we do not try to hide OS indicators.
7. Misuse risks, for example someone installing the app on another person's phone.

12. Technical risks and limitations

Risk	Likelihood	Impact	Mitigation
Alert delayed or lost when the contact's phone is in Doze or Focus	Medium	High	High-priority push, Time Sensitive level, test on real devices
OS stops background location (manufacturer battery rules)	Medium	High	Foreground service, user guidance in setup, test on two brands
Visible indicator or notification reveals the app	High on iOS	High	Design for it, do not rely on hiding it, consult Ben and David
False activations from the trigger	Medium	High	Confirmation window, cancel flow, measured in Spike A and B
Framework plugin unmaintained or incompatible with new	Medium	Medium	Spike before committing; keep

Risk	Likelihood	Impact	Mitigation
OS versions			native code isolated
OS policy changes during the project	Low to medium	Medium	Record the OS versions and access dates for every source
App store policy on disguised apps	Unknown	High	Needs separate review [To verify]; the prototype is not for public release
Prototype mistaken for a guaranteed rescue service	Low	Very high	Clear limitation statements, as set in the brief

13. Decision log entries (brief format)

Date	Question	Evidence	Finding	Decision / implication	Source	Owner
15/10	Can the trigger work in the background ?	Android and iOS docs on foreground services and indicators	Background start restricted; mic blocked or indicated	MVP trigger is foreground only	S3, S5, S6, S21	Ikram
15/10	Can P2P work without a backend?	WebRTC explanation s	Needs signalling, TURN, and push	Do not claim no-backend; MVP uses relay plus push	S18, S19	Ikram
15/10	Flutter or React Native?	2026 comparison articles	Performanc e gap narrow; plugins decide	Spike both before choosing	S15, S16, S17	Ikram
15/10	How do alerts reach a locked phone?	Doze docs, FCM priority, HIG	High priority and Time Sensitive needed	Use both; test on devices	S7, S8, S12	Ikram
15/10	Is ML needed for	LiteRT docs	Possible but not	Rules first; LiteRT as	S13, S14	Ikram

Date	Question	Evidence	Finding	Decision / implication	Source	Owner
	MVP?		required	stretch		

14. Next steps

1. Agree evaluation criteria and weights with the team (section 3 table).
2. Build the three-feature spike in both frameworks (about one sprint).
3. Run Spike A and B for the trigger and log false activations.
4. Build the relay and push prototype and measure scenarios 2, 4 and 5 (section 9).
5. Update this document with measured results, and replace the text diagram with a proper architecture diagram.
6. Hold a joint review with David and Ben on sections 4, 7 and 11.

15. Sources (accessed 15 October 2026)

Official documentation:

- S1: Android Developers, Foreground services. developer.android.com/guide/components/foreground-services
- S2: Android Developers, Changes to foreground services (by Android version). developer.android.com/develop/background-work/services/fgs/changes
- S3: Android Developers, Restrictions on starting foreground services from the background (Android 12). developer.android.com/about/versions/12/foreground-services
- S4: Android Developers, Foreground service types. developer.android.com/develop/background-work/services/fg-service-types
- S5: Android Developers, Foreground services in Android 11 (location, camera, microphone). developer.android.com/about/versions/11/privacy/foreground-services
- S6: Android Developers, Privacy cheat sheet. developer.android.com/privacy/cheat-sheet
- S7: Android Developers, Doze and App Standby. developer.android.com/training/monitoring-device-state/doze-standby
- S8: Firebase, Set and manage Android message priority. firebase.google.com/docs/cloud-messaging/android/message-priority
- S9: Apple, Handling location updates in the background. developer.apple.com/documentation/corelocation/handling-location-updates-in-the-background
- S10: Apple, Requesting authorization to use location services. developer.apple.com/documentation/corelocation/requesting-authorization-to-use-location-services
- S11: Apple, showsBackgroundLocationIndicator. developer.apple.com/documentation/corelocation/cllocationmanager/showsbackgroundlocationindicator
- S12: Apple Human Interface Guidelines, Managing notifications (interruption levels). Confirm the current URL on developer.apple.com/design/human-interface-guidelines

- S13: Google AI Edge, LiteRT overview. ai.google.dev/edge/litert/overview, and TensorFlow Lite page, tensorflow.org/lite
- S14: Google AI Edge, LiteRT repository. github.com/google-ai-edge/LiteRT

Secondary and vendor sources (lower reliability, used with care):

- S15: Adapty, Flutter vs. React Native (updated 9 March 2026). adapty.io/blog/flutter-vs-react-native/
- S16: Techsy, React Native vs Flutter 2026 (updated 12 May 2026). techsy.io/en/blog/react-native-vs-flutter
- S17: Infomaze Elite, React Native vs Flutter 2026. infomazeelite.com
- S18: ToTheNew, Understanding WebRTC: what it is and what it isn't. tothenew.com/blog/?p=74366
- S19: freeCodeCamp, How WebRTC scales: signaling, NAT traversal and the mesh/SFU/MCU tradeoff. freecodecamp.org
- S20: Twilio, Network Traversal Service (vendor claim). twilio.com/en-us/stun-turn
- S21: Y-Security, Detection of audio capture (notes on iOS and Android microphone indicators). redteam.y-security.de/MITRE/detectionstrategies/DET0673/